GAPLYTICS

AI-Adaptive Onboarding Engine

Close the Gap. Own the Role.

Full Technical Documentation · Version 2.0

FastAPI + React 18 + AWS Bedrock (Claude 3 Haiku)

Table of Contents

1. Project Overview ····· 3
2. Architecture & System Design ····· 4
3. Tech Stack ····· 6
4. Setup & Installation ····· 7
5. API Reference ····· 10
6. Skill-Gap Analysis — Deep Dive ····· 12
7. Adaptive Learning Pathway — Deep Dive ····· 15
8. Mock Interview Engine ····· 17
9. Resume Tailor Feature ····· 18
10. HR Portal Features ····· 19
11. Frontend Architecture ····· 20
12. Deployment Guide ····· 22
13. Datasets & Citations ····· 24
14. Troubleshooting ····· 25
15. Validation Metrics ····· 26

1. Project Overview

Gaplytics is a dual-role AI onboarding platform that eliminates generic training paths. HR teams post job descriptions; candidates upload resumes. The AI engine computes precise skill gaps, builds a personalised day-by-day learning roadmap calibrated to the candidate's deadline, and unlocks a score-gated mock technical interview only for candidates who reach 90%+ match.

The platform solves three specific problems that conventional onboarding ignores:

- 82% of new hires lack at least one critical job skill on day one of employment.
- Generic onboarding treats every hire identically, wasting time on skills they already have.
- HR managers have zero real-time visibility into whether a candidate is actually closing their gaps.

1.1 Dual-Role Architecture

Every interaction in Gaplytics happens through one of two lenses:

Role	Entry Point	Core Value
`Candidate / New Hire`	Upload resume PDF + JD	Personalised gap analysis, roadmap, interview
`HR Manager`	Create role from JD	Pipeline visibility, readiness tracking, comparison

1.2 Live Deployment

Service	URL
`Frontend (Vercel)`	https://gaplytics.vercel.app
`Backend API (Render)`	https://gaplytics-fuax.onrender.com
`API Docs (Swagger)`	https://gaplytics-fuax.onrender.com/docs
`Source Code`	https://github.com/balapraharsha/Gaplytics

> **Design Palette**
>
> Chinese Black #0C1519 — page background

> Dark Jungle Green #162127 — cards and panels
> Jet #3A3534 — borders and dividers
> Coffee #724B39 — warm accent and hover states
> Antique Brass #CF9D7B — CTAs, headings, active states

2. Architecture & System Design

Gaplytics is composed of two independently deployed services connected via REST API. There is no shared database — the backend owns all persistent state and the frontend caches candidate-specific data in localStorage.

2.1 High-Level Data Flow

```
CANDIDATE FLOW                          HR FLOW
──────────────────────────────          ──────────────────────────────
1.  Upload resume PDF                   1.  Paste job description
2.  PyMuPDF → raw text                  2.  POST /api/hr/create-role
3.  POST /api/analyze                   3.  Claude extracts requirements
4.  Claude extracts resume skills       4.  role_id + share link returned
5.  Claude extracts JD requirements     5.  HR shares link to candidates
6.  AI semantic skill matching          6.  Candidates complete analysis
7.  Weighted gap score computed         7.  GET /api/hr/role/{id}
8.  Kahn DAG sort → learning order      8.  Pipeline: match%, status, gaps
9.  Deadline bin-packing → schedule     9.  POST /api/hr/compare
10. Score gate: ≥90% → interview       10.  AI side-by-side verdict
```

2.2 Service Boundaries

Layer	Technology	Responsibility
`Frontend SPA`	React 18 + Vite	All UI — candidate and HR flows
`REST API`	FastAPI + Uvicorn	All business logic, AI calls, persistence
`AI Client`	AWS Bedrock (urllib)	Claude 3 Haiku inference — no boto3
`PDF Engine`	PyMuPDF (fitz)	Text extraction from resume and JD PDFs
`HR Store`	hr_data.json + memory	Role and candidate summary persistence
`Candidate State`	Browser localStorage	Analysis cache, roadmap progress, deadline

2.3 Key Design Decisions

- No boto3 dependency — the AWS Bedrock client uses Python's built-in urllib with Bearer token auth, keeping the Docker image small and startup fast.
- Fallback chain — if Claude 3 Haiku returns 403/404, the system automatically tries Claude 3.5 Haiku, then Amazon Titan Text Lite.

- JSON repair — all AI JSON responses pass through a bracket-stack repair function before parsing, preventing crashes from truncated responses.
- Score gate is a hard route guard — the mock interview route checks localStorage on mount and immediately redirects if score < 90. It is never just hidden.
- Semantic matching runs as a single batch AI call — all unmatched JD requirements are sent together, not one by one, to minimise latency.

3. Tech Stack

3.1 Backend Dependencies

Package	Version	Purpose
`fastapi`	≥0.110.0	REST API framework
`uvicorn[standard]`	≥0.27.0	ASGI production server
`python-multipart`	≥0.0.9	Multipart form data for file uploads
`pymupdf`	≥1.23.0	PDF text extraction (fitz module)
`python-dotenv`	≥1.0.0	Load .env environment variables
`pydantic`	≥2.0.0	Request/response model validation
`mangum`	≥0.17.0	Optional AWS Lambda handler

3.2 Frontend Dependencies

Package	Purpose
`react@18, react-dom@18`	Core UI framework
`react-router-dom@6`	SPA routing — 10 routes
`vite@5`	Build tool and dev server
`tailwindcss@3`	Utility-first CSS
`framer-motion`	Physics-based animations and page transitions
`recharts`	RadarChart for 9-dimension interview results
`react-hot-toast`	Toast notification system
`axios`	HTTP client for all API calls
`react-dropzone`	PDF drag-and-drop upload zones
`highlight.js`	Code syntax highlighting in interview questions

4. Setup & Installation

4.1 Prerequisites

- Python 3.11 or higher
- Node.js 18 or higher
- AWS Bedrock long-term API key (see 4.2)
- Git

4.2 AWS Bedrock API Key

1. Go to https://console.aws.amazon.com → search Amazon Bedrock
2. Left sidebar → API keys → Generate long-term API key
3. Copy the key (starts with ABSKQm...)
4. Go to Bedrock → Model access → Enable Claude 3 Haiku

> **Important**
> The key must be from us-east-1 region unless you change AWS_REGION in .env.
> If Claude 3 Haiku is not enabled, all 3 models in the fallback chain will return 403.

4.3 Clone & Backend Setup

```
git clone https://github.com/balapraharsha/Gaplytics.git
cd Gaplytics/backend
cp .env.example .env

# Edit .env with your values:
AWS_BEARER_TOKEN_BEDROCK=your_key_here
AWS_REGION=us-east-1
BEDROCK_MODEL=anthropic.claude-3-haiku-20240307-v1:0
FRONTEND_URL=http://localhost:5173
SEMANTIC_THRESHOLD=0.75

pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

Verify at: http://localhost:8000/api/health

```
{"status": "ok", "version": "5.0.0", "configured": true}
```

4.4 Frontend Setup

```
cd ../frontend
cp .env.example .env
# VITE_API_URL=http://localhost:8000  (default — no change needed)

npm install
npm run dev
# Opens at http://localhost:5173
```

4.5 Environment Variables Reference

Variable	Required	Default	Description
`AWS_BEARER_TOKEN_BEDROCK`	Yes	—	AWS Bedrock long-term API key
`AWS_REGION`	No	us-east-1	AWS region for Bedrock endpoint
`BEDROCK_MODEL`	No	anthropic.claude-3-haiku-20240307-v1:0	Primary model ID
`FRONTEND_URL`	No	http://localhost:5173	CORS allowed origin
`SEMANTIC_THRESHOLD`	No	0.75	Skill matching sensitivity (0.0–1.0)
`VITE_API_URL`	Yes (FE)	http://localhost:8000	Backend base URL

4.6 Docker

```
cd backend
docker build -t gaplytics-backend .
docker run -p 8000:8000 --env-file .env gaplytics-backend
```

5. API Reference

All endpoints are available at the Render backend URL. Full Swagger UI is at /docs. All POST requests use JSON body unless otherwise noted.

5.1 Core Endpoints

Method	Endpoint	Description
GET	/api/health	Health check — returns version, model, configured status
GET	/api/ping	Live Bedrock ping — returns GAPLYTICS_OK
POST	/api/analyze	Full resume + JD analysis (multipart form)
POST	/api/chat	AI coach — context-aware response
POST	/api/tailor-resume	AI resume rewriting with gap skills
POST	/api/interview/generate	Generate 12 questions across 6 types
POST	/api/interview/evaluate	9-dimension answer evaluation
POST	/api/interview/feedback	Single question practice feedback

5.2 HR Endpoints

Method	Endpoint	Description
POST	/api/hr/create-role	Create role from JD — returns role_id + candidate URL
GET	/api/hr/roles	List all HR roles
GET	/api/hr/role/{role_id}	Role detail + full candidate pipeline
GET	/api/hr/candidates/{role_id}	Candidate summaries for a role
POST	/api/hr/compare	AI side-by-side candidate comparison

5.3 /api/analyze Request Format

```
POST /api/analyze
Content-Type: multipart/form-data

Fields:
  resume_file   — PDF file (required)
  jd_text       — Job description text (required)
  deadline      — Date string YYYY-MM-DD (optional)
  daily_hours   — Float, hours per day (optional, default 2.0)
  role_id       — UUID string (optional, links candidate to HR role)
```

5.4 /api/analyze Response Shape

```
{
  "gap_result": {
    "candidate_name": "string",
    "target_role": "string",
    "overall_match_percentage": 72.5,
    "score_gate": "feedback" | "mock_interview",
    "known_skills": [...],
    "gap_skills": [...],
    "rejection_reasons": [...]
  },
  "pathway": {
    "total_hours": 18.0,
    "daily_schedule": { "days": [...], "total_days": 90 },
    "reasoning_trace": [...],
    "nodes": [...]
  },
  "modules_detail": [...]
}
```

6. Skill-Gap Analysis — Deep Dive

The skill-gap engine is the analytical core of Gaplytics. It transforms two unstructured text inputs — a resume PDF and a job description — into a precise numerical gap score, a prioritised list of missing skills, and a weighted overall match percentage. The logic lives in parser.py and skill_gap.py.

6.1 Stage 1 — PDF Text Extraction

Both the resume and the job description enter the pipeline as raw text extracted by PyMuPDF:

```
# parser.py
def extract_text_from_pdf(file_bytes: bytes) -> str:
    doc = fitz.open(stream=file_bytes, filetype='pdf')
    text = '\n'.join(page.get_text() for page in doc)
    doc.close()
    return text
```

Text is capped at 8,000 characters before being sent to the AI model. This keeps token costs low while retaining all meaningful content from typical resume and JD lengths.

6.2 Stage 2 — AI Skill Extraction

The raw text is sent to AWS Bedrock (Claude 3 Haiku) with two separate structured prompts. Each returns a strict JSON array validated by Pydantic v2:

```
# Resume skill output shape
[{
  "name": "python",
  "category": "programming_language",
  "proficiency_level": 7,    // 1-10 scale
  "years_experience": 3.0,
  "evidence": "Built REST APIs with FastAPI"
}]

# JD requirement output shape
[{
  "name": "python",
  "required_level": 9,       // 1-10 scale
  "is_mandatory": true,
  "job_market_relevance_pct": 82
}]
```

6.3 Stage 3 — Skill Name Normalisation

All skill names pass through normalize_skill_name() before any comparison. This function lowercases, strips whitespace, and resolves a hand-curated alias map:

```
ALIAS = {
  "js": "javascript",  "ts": "typescript",  "py": "python",
  "k8s": "kubernetes", "ml": "machine learning",
  "tf": "tensorflow",  "node": "node.js",
  "reactjs": "react",  "aws": "amazon web services",
  # ...20+ more aliases
}
```

6.4 Stage 4 — Three-Stage Keyword Matching

For each JD requirement, the engine attempts to find the best matching resume skill:

Stage	Method	Threshold	Example
`1 — Exact/Alias`	normalize_skill_name() equality	score = 1.0	js → javascript ✓
`2 — Substring`	One name contains the other	score = 0.85	python 3 → python ✓
`3 — Word Overlap`	Shared word ratio ≥ 50%	score ≥ 0.50	machine learning eng → ml ✓

6.5 Stage 5 — AI Semantic Fallback

For skills that pass through all three keyword stages without a match, a single AI call handles the remaining unmatched requirements. This eliminates the most common source of false negatives — wording mismatches:

JD Requirement	Resume Skill	Without AI	With AI Semantic
`cloud platforms`	AWS (EC2, S3, Lambda)	✗ No match	✓ Matched
`end-to-end ML`	TensorFlow pipelines	✗ No match	✓ Matched
`NLP & semantic analysis`	LLM + NLP work	✗ No match	✓ Matched
`containerisation`	Docker, Kubernetes	✗ No match	✓ Matched

6.6 Stage 6 — Weighted Gap Score

The final match percentage uses mandatory-2x weighting to reflect real-world hiring priorities:

```
total_weight   = sum(2 if r.is_mandatory else 1 for r in jd_requirements)
matched_weight = 0.0

for req in jd_requirements:
    c_level   = find_candidate_level(req, resume_skills)
    match_pct = min(100, (c_level / req.required_level) * 100)
    weight    = 2 if req.is_mandatory else 1
    matched_weight += (match_pct / 100) * weight

overall_match = round((matched_weight / total_weight) * 100, 1)
```

6.7 Gap Priority Classification

Priority	Condition	Impact
`critical`	is_mandatory = true AND gap_score ≥ 5	Shown first, drives rejection reasons
`high`	gap_score ≥ 3	High priority in roadmap scheduling
`medium`	gap_score ≥ 1	Included in roadmap
`low`	gap_score = 0 (candidate meets level)	Shown as known skill

7. Adaptive Learning Pathway — Deep Dive

After gap analysis, the pathway generator builds a personalised day-by-day learning schedule. Four original algorithms run in sequence inside adaptive.py.

7.1 Algorithm 1 — Graph-Based Prerequisite DAG

The course catalog (50+ modules) is represented as a directed acyclic graph. Each CourseModule has a prerequisites[] array. Relevant modules are selected by matching gap skill names, then transitive prerequisites are recursively expanded:

```
def add_prerequisites(module_id, visited):
    if module_id in visited: return
    visited.add(module_id)
    module = module_map[module_id]
    for prereq_id in module.prerequisites:
        relevant_ids.add(prereq_id)
        add_prerequisites(prereq_id, visited)
```

Module selection uses 4 stages: exact skill name match → word overlap (≥40%) → title keyword match → AI semantic fallback for remaining unmatched gaps.

7.2 Algorithm 2 — Kahn's Topological Sort (BFS)

The selected modules are ordered using Kahn's algorithm with in-degree tracking. This produces a valid linear learning order from foundational to advanced skills:

```
# Build in-degree map
in_degree = {m: 0 for m in module_ids}
for m in modules:
    for prereq in graph[m.id]:
        in_degree[m.id] += 1

# BFS from zero-in-degree nodes
queue = [m for m in in_degree if in_degree[m] == 0]
sorted_ids = []
while queue:
    node = queue.pop(0)
    sorted_ids.append(node)
    for child in dependents[node]:
        in_degree[child] -= 1
        if in_degree[child] == 0:
            queue.append(child)

# Cycle detection
if len(sorted_ids) < len(module_ids):
```

```
    raise ValueError('Cycle detected in prerequisite graph')
```

7.3 Algorithm 3 — Deadline-Aware Bin-Packing

Modules are packed into day slots respecting the daily_hours budget. If required_daily > budget, the system auto-adjusts with a 10% ceiling:

```
required_daily = total_content_hours / total_days
if required_daily > daily_hours:
    daily_hours = round(min(required_daily * 1.1, 12), 1)  # max 12h/day
    warning = f'Adjusted to {daily_hours}h/day to meet deadline'

# Last 2 days always reserved
reserved_start = deadline_date - timedelta(days=1)  # final_review
# deadline_date itself = mock_test
```

7.4 Algorithm 4 — Priority-Weighted Scheduling

Critical and high-priority gap modules are scheduled in the first 60% of days. Review and rest days alternate in the remaining window. The pattern is: review → rest → review → rest → final_review → mock_test.

7.5 Day Type Reference

Day Type	Content	Visual
`study`	One or more course modules within daily_hours budget	Brass checkbox cards
`review`	No modules — revisit weak areas from previous days	Muted border
`rest`	No modules — full rest day	Green muted border
`final_review`	2h — review everything before the mock test	Bright brass border
`mock_test`	1h — deadline day, mock test readiness check	Elite gold border

8. Mock Interview Engine

The mock interview is locked behind a score gate. Candidates must reach overall_match_percentage ≥ 90 to unlock it. This gate is enforced at the route level in AuthorizedInterview.jsx, not just via UI visibility.

8.1 Question Generation

12 questions are generated across 6 types based on the candidate's specific gap skills and target role:

Question Type	Tests
`scenario_decision`	Architecture and design decision-making under constraints
`debug_the_code`	Identify and explain bugs in provided code snippets
`fix_the_code`	Write corrected code in the answer textarea
`code_review`	Identify issues in a code review scenario
`log_detective`	Diagnose system issues from log output
`complexity_analysis`	Analyse time/space complexity and suggest improvements

8.2 9-Dimension Evaluation

Every answer set is evaluated by Claude across 9 dimensions, each scored 0–10:

Dimension	What It Measures
`decision_making`	Quality of architectural and design choices
`debugging_ability`	Accuracy and approach to bug identification
`code_correctness`	Whether code fixes are syntactically and logically correct
`code_quality`	Readability, naming, structure, and best practices
`incident_diagnosis`	Log reading and root cause analysis ability
`algorithmic_thinking`	Problem decomposition and algorithm selection
`communication_clarity`	Ability to explain technical concepts clearly
`adaptability_under_pressure`	Performance under timed, high-stakes conditions
`technical_depth`	Depth of domain knowledge demonstrated

8.3 Verdict Thresholds

Verdict	Score Range	Meaning
`HIRE_READY`	≥ 8.5 / 10	Ready for immediate onboarding
`STRONG_CANDIDATE`	7.0 – 8.4	Strong — minor gaps remain
`NEEDS_MORE_PREP`	< 7.0	More preparation required before hiring

9. Resume Tailor Feature

Candidates with scores between 60% and 89% see the Resume Tailor card on their dashboard. This feature generates a complete, ATS-optimised resume that weaves in their gap skills as in-development competencies.

9.1 Trigger Conditions

- Score ≥ 60% AND score < 90% — shown on Dashboard
- Score < 60% — not shown (candidate needs more foundational work first)
- Score ≥ 90% — not shown (candidate goes to interview instead)

9.2 Generated Resume Sections

The backend prompt explicitly requires 8 complete sections in order:

5. Professional Summary — rewritten for the specific target role
6. Core Skills — all existing skills preserved
7. Skills In Development — gap skills added with one-line context each
8. Professional Experience — 2 roles with action-verb bullet points using JD keywords
9. Education — degree and relevant coursework
10. Projects & Achievements — portfolio items with impact framing
11. Certifications & Learning — active learning pathways mentioned

9.3 Download as PDF

The download function converts markdown to styled HTML using a markdownToHtml() converter, injects it into a print-ready HTML template with the Deep Maritime palette's print-safe equivalent (white background, dark text), opens the document in a new browser tab, and auto-triggers the print dialog. The user saves as PDF from the browser.

10. HR Portal Features

10.1 Role Creation

HR managers paste a job description (or upload a JD PDF). Claude extracts all required skills with required_level and is_mandatory flags. The system returns a role_id and a shareable candidate URL.

10.2 Candidate Pipeline

The pipeline table shows every candidate who completed an analysis using the role's share link. Columns are sortable by name, match %, status, progress %, and days to ready. Top gaps are shown as chips. Candidates can be selected for side-by-side comparison.

10.3 Team Readiness Bar

The readiness bar shows the average match % across all candidates in the pipeline, with colour-coded tier badges: ELITE (≥90%), STRONG (≥70%), DEVELOPING (≥60%), NEEDS WORK (<60%), and BEHIND (status === 'behind').

10.4 Candidate Progress View

Clicking View Progress on any candidate opens a dedicated page showing their investment tracker: match score ring, roadmap completion bar, hours invested, days to ready, pace status, top skill gap bars, and a readiness timeline with 4 steps.

10.5 Compare View

Select exactly 2 candidates and click Compare. Claude generates a side-by-side verdict in 2 sentences, recommending one over the other. A skill comparison table shows Has/Gap status for every required skill.

11. Frontend Architecture

11.1 Route Map

Path	Component	Access
`/`	Landing.jsx	Public
`/candidate`	CandidateUpload.jsx	Public
`/candidate/dashboard`	Dashboard.jsx	Requires localStorage analysis
`/candidate/roadmap`	DeadlineRoadmap.jsx	Requires localStorage analysis
`/candidate/interview`	AuthorizedInterview.jsx	Requires score ≥ 90
`/candidate/results`	InterviewResults.jsx	Requires localStorage result
`/hr`	HRDashboard.jsx	Public
`/hr/role/:role_id`	RoleDetail.jsx	Public
`/hr/compare`	CompareView.jsx	Requires route state
`/hr/candidate/:id`	CandidateProgressView.jsx	Requires localStorage candidate

11.2 Key Components

Component	Purpose
`ChatCoach.jsx`	FAB + slide-out chat panel, sends analysis context to /api/chat
`ResumeTailor.jsx`	AI resume rewriting for 60-89% scores, PDF download
`ReasoningTrace.jsx`	Collapsible AI reasoning steps panel
`InteractiveFeedback.jsx`	Gap breakdown with tips and progress-to-90 indicator
`LeafBackground.jsx`	Ambient particle animation on all pages
`Navbar.jsx`	Glass nav with breadcrumbs, role switcher, GitHub link

11.3 localStorage Keys

Key	Content	Set By
`gaplytics_analysis`	Full analysis result (gap_result + pathway + modules_detail)	CandidateUpload.jsx
`gaplytics_deadline`	Deadline date string YYYY-MM-DD	CandidateUpload.jsx
`gaplytics_progress`	Module completion map {date: [module_ids]}	DeadlineRoadmap.jsx
`gaplytics_interview_result`	Full interview evaluation object	MockInterview.jsx
`gaplytics_interview_questions`	Array of 12 question objects	MockInterview.jsx
`gaplytics_interview_answers`	Array of answer objects	MockInterview.jsx
`gaplytics_hr_viewing_candidate`	CandidateSummary object for HR view	RoleDetail.jsx
`gaplytics_hr_viewing_role`	HRRole object for context in progress view	RoleDetail.jsx

12. Deployment Guide

12.1 Frontend →Vercel

12. Push repository to GitHub
13. Go to vercel.com →New Project →Import from GitHub
14. Select balapraharsha/Gaplytics
15. Set Root Directory to frontend
16. Framework Preset: Vite (auto-detected)
17. Add environment variable: VITE_API_URL = your Render backend URL
18. Click Deploy — auto-redeploys on every push to main

> **Auto-Deploy**
> Every git push to main triggers a Vercel rebuild automatically.
> Build time is approximately 45 seconds for the frontend.

12.2 Backend →Render

19. Go to render.com →New →Web Service
20. Connect balapraharsha/Gaplytics repository
21. Set Root Directory to backend
22. Build Command: pip install -r requirements.txt
23. Start Command: uvicorn main:app --host 0.0.0.0 --port $PORT
24. Add all environment variables from .env.example
25. Click Deploy

> **Free Tier Note**
> Render free tier services sleep after 15 minutes of inactivity.
> First request after sleep takes 30-60 seconds.
> Visit /api/health before a demo to wake the service up.

12.3 Post-Deployment Checklist

- Visit https://gaplytics-fuax.onrender.com/api/health — verify configured: true
- In Vercel, confirm VITE_API_URL points to Render URL (not localhost)

- Redeploy Vercel after updating any environment variables
- Enable Claude 3 Haiku in AWS Console → Bedrock → Model access
- Test full flow: upload resume → get analysis → check roadmap → verify dashboard

13. Datasets & Citations

Dataset	Source	Usage in Gaplytics
O*NET 30.2 Database	onetcenter.org/db_releases.html	Powers job_market_relevance_pct across all 50+ course modules. Skill taxonomy categories injected into AI extraction prompts.
Kaggle Resume Dataset	kaggle.com/datasets/snehaanbhawal/resume-dataset	Parser validation across 10 labelled resumes. Test fixtures in backend/tests/fixtures/.
Kaggle Jobs & JD Dataset	kaggle.com/datasets/kshitizregmi/jobs-and-job-description	JD extraction validation across 5 domains: Software, Healthcare, Retail, Warehouse, Finance.

Citation

O*NET 30.2 — U.S. Department of Labor / Employment and Training Administration.
Anbhawal, Sneha. Resume Dataset. Kaggle, 2020.
Regmi, Kshitiz. Jobs and Job Description. Kaggle, 2022.

14. Troubleshooting

Error / Symptom	Root Cause	Fix
`Analysis failed. Please try again`	Backend not reachable from frontend	Check VITE_API_URL in Vercel points to Render URL, not localhost
`configured: false in health check`	AWS_BEARER_TOKEN_BEDROCK missing or blank	Add the key to Render environment variables — no spaces around =
`All models returning 403`	API key expired or model access not enabled	Regenerate key in AWS Console → Bedrock →API keys. Enable Claude 3 Haiku in Model access.
`No skills found in resume`	Scanned PDF (image-based)	Use a text-based PDF — PyMuPDF cannot OCR images
`Invalid JSON from Bedrock`	Response truncated mid-JSON	Transient — retry. Max tokens set to 6000 to minimise recurrence.
`CORS error in browser console`	Backend not running or wrong port	Start backend on 8000 and verify VITE_API_URL matches
`Roadmap shows only review/rest days`	Content hours fit in very few days	Increase daily_hours or extend deadline — 18h content / 2h/day = 9 study days
`Score gate not unlocking interview`	Score < 90 or stale localStorage	Clear localStorage and re-run analysis
`HR role creation returns 502`	Bedrock 403 — same as above	Refresh API key and update Render env var

15. Validation Metrics

Internal metrics used to validate the accuracy and efficiency of the Gaplytics engine:

Metric	Value	Method	Target
`Skill Extraction Precision`	87%	10 labelled Kaggle resumes vs Claude output	≥ 85%
`Semantic Match Recall`	83%	20 manually built synonym pairs	≥ 80%
`Training Time Reduction`	74%	(catalog_hrs − pathway_hrs) / catalog_hrs	≥ 70%
`Cross-Domain Coverage`	5×	Validated across Software, Healthcare, Retail, Warehouse, Finance	≥ 4 domains
`Schedule Feasibility`	100%	20 random (pathway, deadline) pairs	100%
`JSON Parse Success Rate`	98%	500 consecutive AI calls with repair enabled	≥ 95%

GAPLYTICS — Full Technical Documentation — Version 5.0

Built for the Gaplytics Hackathon · Powered by AWS Bedrock (Claude 3 Haiku by Anthropic)

github.com/balapraharsha/Gaplytics · gaplytics.vercel.app